frontend

git

Git без магии

История проекта, ветки и Pull Request

Вторая пара

Git — для слабых духом?

НАСТОЯЩИЙ РАЗРАБОТЧИК

project
project-final
project-final-2
project-final-REAL
project-final-VERY-FINAL
project-final-VERY-FINAL-fixed

А потом непонятно, где рабочая версия и что изменилось.

Почему копии папок не работают

- Название папки не объясняет, что именно изменилось
- Версии занимают место и быстро расходятся
- Два человека не могут нормально объединить изменения
- Откат превращается в поиск подходящего архива

Главная проблема: история проекта существует только в голове автора.

Цели пары

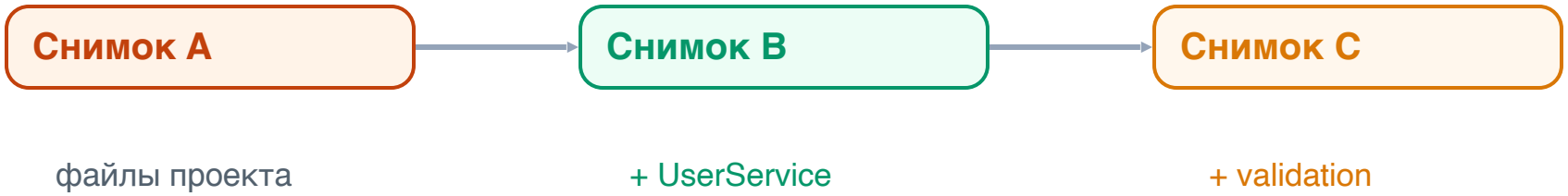
- Понять модель Git: рабочие файлы, индекс и история
- Разобраться с commit, branch, merge и remote
- Отделить Git от GitHub и GitLab
- Понять, как устроена сдача через Pull Request

Что такое система контроля версий

Система контроля версий хранит историю изменений файлов и помогает нескольким людям работать над одним проектом.

- Можно увидеть, кто, когда и зачем изменил файл
- Можно сравнить версии и вернуться к прошлому состоянию
- Можно вести несколько линий разработки и объединять их

Git хранит снимки проекта



Каждый commit указывает на состояние проекта и на предыдущий commit.

Git оптимизирует хранение, но мыслить удобнее именно снимками.

Три области Git



Файлы, которые вы сейчас редактируете

Изменения, выбранные для следующего commit

Сохранённые commit и связи между ними

Базовый цикл работы

1

Изменить файлы

редактор / IDE

2

Проверить состояние

git status

3

Выбрать изменения

git add

4

Сохранить снимок

git commit

Commit остаётся локально. Отправка на сервер происходит отдельно.

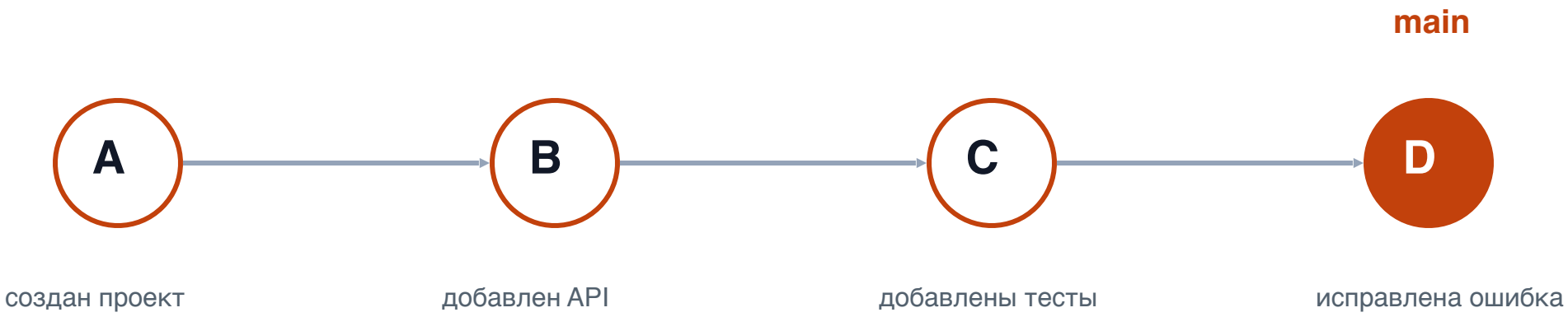
Commit

Commit — сохранённый снимок проекта с автором, временем, сообщением и ссылкой на предыдущий commit.

`a83f21c` **Добавить проверку пустого имени**

Один commit должен выражать одну законченную мысль.

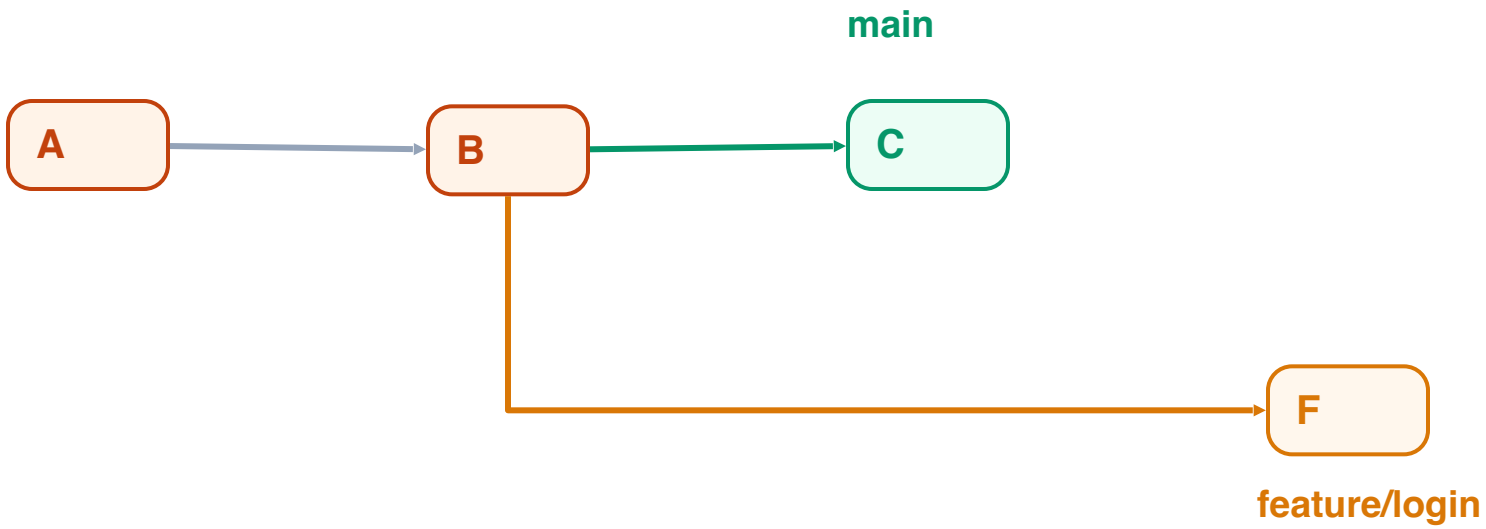
История commit



История Git — граф, а не набор папок и не простой список файлов.

Ветка

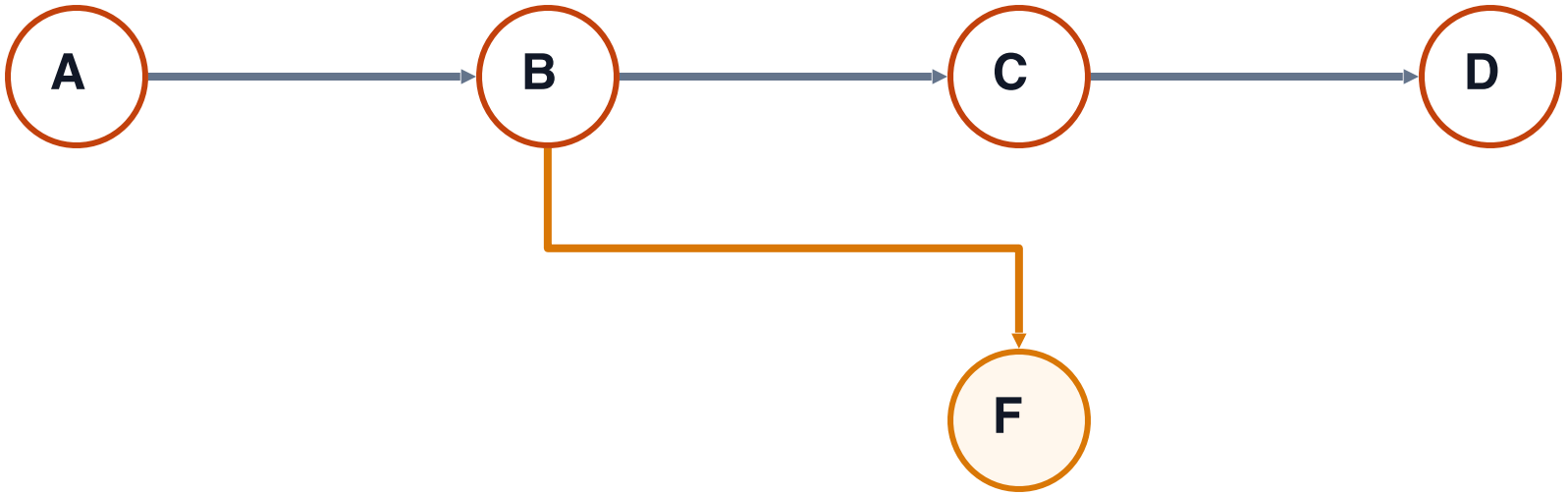
Ветка — подвижное имя, которое указывает на последний commit своей линии разработки.



Создание ветки не копирует весь проект.

Аналогия с железной дорогой

История проекта



commit — станция

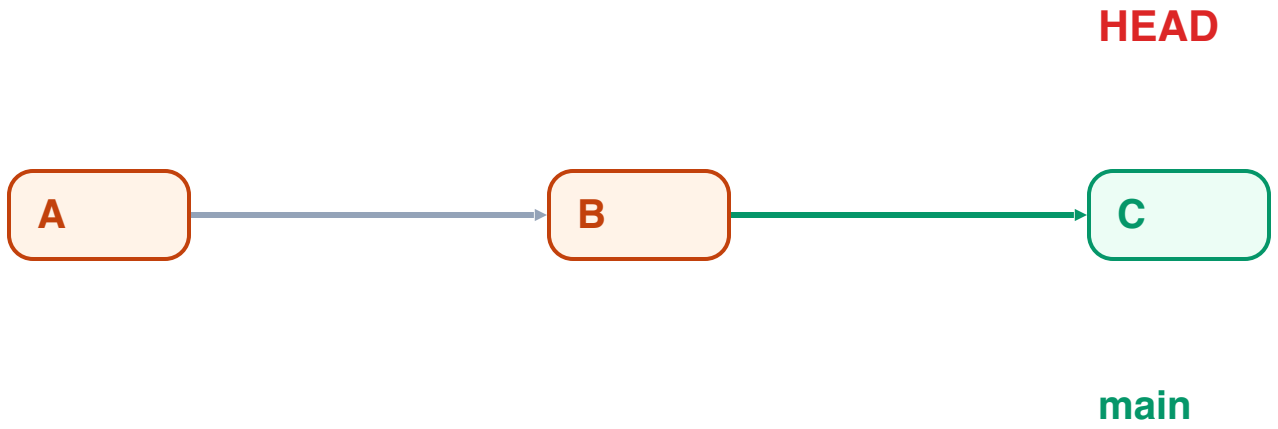
branch — отдельный путь

merge — стрелка, которая соединяет пути

HEAD показывает, на каком пути и у какой станции мы сейчас находимся.

HEAD

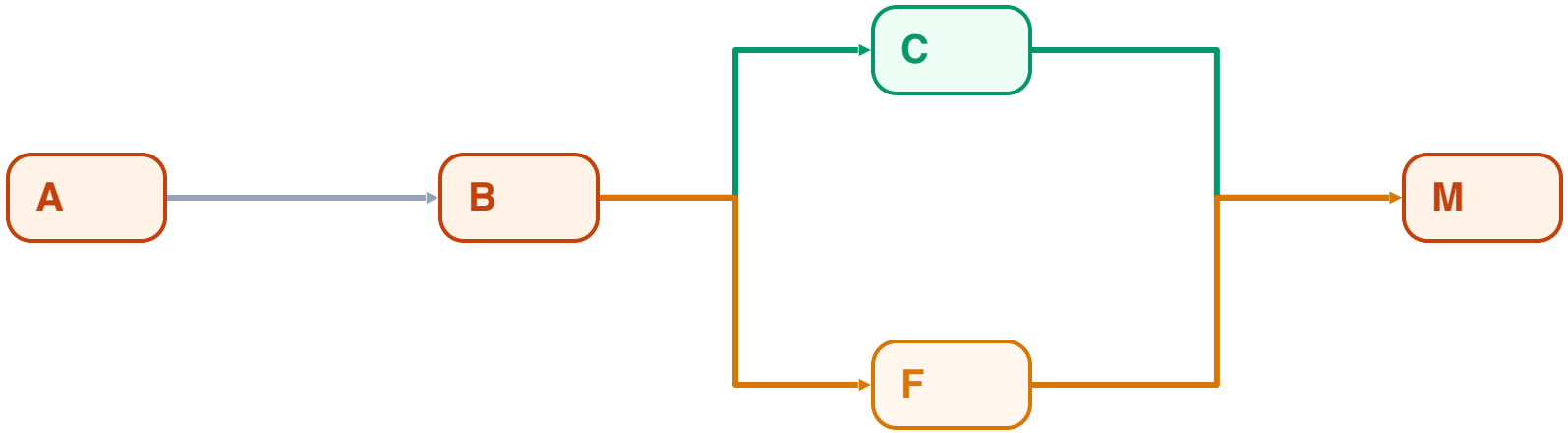
HEAD — указатель на текущую ветку или конкретный commit.



git switch меняет текущую ветку и содержимое рабочей папки.

Merge

Merge объединяет изменения двух линий разработки.



Git объединяет автоматически, если изменения не противоречат друг другу.

Конфликт слияния

Конфликт возникает, когда Git не может сам решить, какой вариант оставить.

main

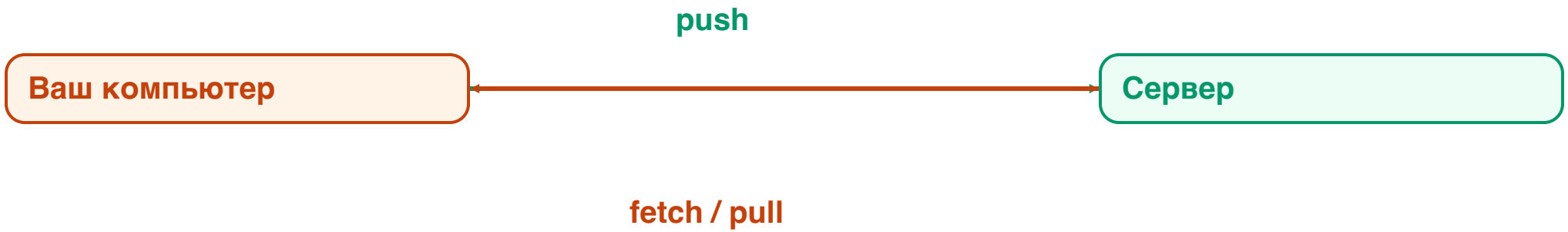
```
server.port=8080
```

feature

```
server.port=9090
```

Разработчик выбирает результат, проверяет проект и завершает merge commit.

Локальный и удалённый репозиторий



Git работает локально. Сервер нужен для обмена, резервной копии и совместной работы.

Git, GitHub и GitLab

Git	Программа и формат репозитория	commit, branch, merge, история
GitHub	Сервис для размещения Git-репозиториев	Pull Request, Issues, Actions
GitLab	Похожая платформа и сервер	Merge Request, Issues, CI/CD

Pull Request принадлежит платформе, а не самому Git.

Clone, fetch, pull и push

git clone	создать локальную копию удалённого репозитория
git fetch	получить новые данные, не меняя рабочую ветку
git pull	получить данные и сразу объединить их с текущей веткой
git push	отправить локальные commit на удалённый сервер

Clone и Fork

CLONE

Копия репозитория с сервера на ваш компьютер

git clone URL

FORK

Отдельная копия чужого репозитория внутри GitHub или GitLab

операция платформы

На курсе вы сдаёте работы в своём репозитории. Fork для этого не нужен.

Pull Request

Pull Request — предложение объединить одну ветку с другой, сопровождаемое обсуждением и проверками.



PR не копирует файлы отдельно. Он показывает разницу между ветками.

Жизненный цикл Pull Request



Ветка

работа над ДЗ

Push

ветка на GitHub

PR

сравнение с main

Review

замечания

Merge

после принятия

Исправления после review добавляются новыми commit в ту же ветку и тот же PR.

Code Review

- Reviewer смотрит разницу между ветками, а не весь проект заново
- Замечание привязывается к конкретной строке или общему решению
- Автор исправляет код, делает commit и отправляет его в ту же ветку
- Обсуждение закрывается после исправления или договорённости

Цель review — найти проблему до merge и передать контекст, а не поставить автора в тупик.

Классический Git Flow

main	Готовое состояние и выпущенные версии	
develop	Интеграция изменений для следующего релиза	
feature/*	создаётся от develop	**возвращается в develop**
release/*	создаётся от develop	**возвращается в main + develop**
hotfix/*	создаётся от main	**возвращается в main + develop**

В оригинальной схеме ветка main называлась master.

Когда нужен Git Flow

ПОДХОДИТ

- Продукт выпускается версиями
- Нужно поддерживать несколько версий одновременно
- Есть отдельная подготовка релиза

МОЖЕТ БЫТЬ ИЗБЫТОЧЕН

- Команда использует непрерывную доставку
- Изменения быстро попадают в production
- Нет параллельной поддержки выпусков

На курсе используем упрощённый flow: main + отдельная ветка каждого ДЗ + Pull Request.

Правила при разработке и сдаче ДЗ

- ДЗ выполняется в отдельной ветке и сдается в формате ссылки на Pull Request (PR/ПР). ПР создается из ветки ДЗ в основную ветку
- Каждый ПР содержит все файлы задания и только их. Даже если ДЗ связаны
- При возврате задания с доработки новый ПР создавать не нужно. Одно ДЗ = один ПР

Если сдали не ссылку на ПР или в ПР есть лишние или недостающие файлы, отправляю сдавать в нужном формате

ПР можно мерджить (merge) после того, как работа принята

Что лежит в репозитории

project/

```
├── src/
├── tests/
├── package.json
├── README.md
├── .gitignore
└── .git/
```

- README объясняет назначение и запуск проекта
- .gitignore исключает временные файлы и секреты
- .git содержит историю и служебные данные Git

.git нельзя отправлять отдельно, редактировать вручную или вкладывать в другой репозиторий.

Сообщения commit: повелительное наклонение

ХОРОШО

Добавить проверку пустого имени
Исправить расчёт итоговой суммы
Добавить тесты регистрации

ПЛОХО

fix
update
final final
asdasd

Формулировка звучит как команда кодовой базе: добавить, исправить, удалить.

Типичные ошибки

- Работа сделана прямо в main
- git add . добавил IDE, build и секреты
- Один commit содержит несколько несвязанных заданий
- После review создан новый PR вместо обновления старого
- Локальная ветка есть, но push на сервер не выполнен

git status и вкладка Files changed позволяют заметить большинство проблем до сдачи.

Мини-задача: подготовить сдачу

Ситуация

Вы клонировали репозиторий, изменили UserService.js и добавили UserServiceTest.js. В main уже появились изменения преподавателя.

Составьте порядок действий от текущего состояния до ссылки на Pull Request.

- Какие проверки выполнить до commit?
- Когда обновить main?
- Какие ветки выбрать в Pull Request?

5 минут в парах

Разбор мини-задачи

1. Проверить git status и diff
2. Временно убрать незаконченные изменения: git stash
3. Обновить main и создать от него ветку homework-1
4. Вернуть изменения: git stash pop
5. Добавить нужные файлы, проверить staged diff и сделать commit
6. **Push ветки и Pull Request: homework-1 в main**

Практика в терминале

```
git clone URL
git switch -c homework-0
git status
git add README.md
git commit -m "Добавить ФИО и группу"
git push -u origin homework-0
```

- Проверить ветку
- Проверить staged diff
- Открыть PR в main
- Скопировать ссылку на PR

Команды выполняем вместе. Ошибки разбираем на месте.

Что нужно унести с пары

- Git хранит историю снимков проекта локально
- git add выбирает изменения, git commit сохраняет снимок
- Ветка указывает на линию разработки, merge объединяет линии
- GitHub и GitLab размещают репозитории и добавляют Pull Request
- Одно ДЗ выполняется в отдельной ветке и сдаётся одним PR

Вопросы?

Следующий шаг: нулевая работа и первый Pull Request